LAPORAN TUGAS
PEMOGRAMAN BERORIENTASI OBJEK
SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS
MENGGUNAKAN ENAM STRUKTUR DATA BERBASIS JAVA



Disusun Oleh:

Kardila Varani Barutu	2509116009
Zahra Maysita	2509116015
Mohammed Rifqy Syahdi	2509116037
Nabila Fitriani	2509116063

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS MULAWARMAN

2026

DAFTAR ISI

DAFTAR ISI ..... **ii**
DAFTAR GAMBAR ..... **iv**
DAFTAR TABEL ..... **v**
BAB I PENDAHULUAN **1**
1.1 Deskripsi Studi Kasus 1
1.2 Tujuan 1
BAB II PERANCANGAN DAN PENERAPAN STRUKTUR DATA **2**
2.1 Array 1 Dimensi (Array 1D) 2
2.2 Array 2 Dimensi (Array 2D) 2
2.3 Array 3 Dimensi (Array 3D) 2
2.4 ArrayList 3
2.5 Stack (LIFO) 3
2.6 Queue (FIFO) 3
2.7 Hubungan Keenam Struktur Data 3
BAB III PENJELASAN SOURCE CODE **5**
3.1 Package, Import, dan Method Main 5
3.2 Deklarasi Struktur Data dan Variabel 5
3.3 Perulangan Menu Utama dan Switch 5
3.4 Menu 1 dan Menu 2 6
3.5 Menu 3: Lokasi Buku 6
3.6 Menu 4: Antrean Mahasiswa 6
3.7 Menu 5: Catat Peminjaman dan Statistik 7
3.8 Menu 6: Pembatalan Transaksi 7
3.9 Menu 7 dan Default 7
3.10 Rincian Kode 8
BAB IV DOKUMENTASI DAN HASIL PENGUJIAN **17**
4.1. Menampilkan Array 1 Dimensi (Kategori Buku) 17
4.2. Memasukkan Data ke ArrayList (Tambah Buku) 17
4.3. Menampilkan Isi ArrayList (Daftar Buku) 18
4.4. Menampilkan Array 3 Dimensi (Lokasi Buku) 19
4.5. Tampilan Queue (Mengelola Antrean Mahasiswa) 19
4.6. Menampilkan Array 2 Dimensi dan Stack (Pencatatan Peminjaman Buku) 21

4.7. Tampilan Stack (Undo Peminjaman Buku) 21
4.8. Tampilan Validasi Tiga Kali Kesalahan Input pada Fitur Peminjaman 22
4.9. Keluar dari Program 23
BAB V KESIMPULAN 24
5.1 Kesimpulan 24
LAMPIRAN 25

DAFTAR GAMBAR

Gambar 4. 1 Tampilan Array 1 Dimensi (Kategori Buku) 17
Gambar 4. 2 Tampilan Penambahan Data Buku ke ArrayList 18
Gambar 4. 3 Tampilan Isi ArrayList (Daftar Buku) 18
Gambar 4. 4 Tampilan Pengisian Data Array 3 Dimensi (Lokasi Buku) 19
Gambar 4. 5 Tampilan Penambahan Data ke Queue (Antrean Mahasiswa) 20
Gambar 4. 6 Tampilan Pemanggilan Data dari Queue (Antrean Mahasiswa) 20
Gambar 4. 7 Tampilan Pencatatan Peminjaman Buku Menggunakan Array 2 Dimensi dan Stack 21
Gambar 4. 8 Tampilan Penghapusan Data dari Stack (Undo Peminjaman) 22
Gambar 4. 9 Validasi Tiga Kali Kesalahan Input pada Fitur Peminjaman 23
Gambar 4. 10 Tampilan Keluar dari Program 23

DAFTAR TABEL

Tabel 3. 1 Ringkasan Variabel, Perintah, dan Alur 8
Tabel 3. 2 Penjelasan Kode Pada SistemPerpustakaan.java 9

BAB I
PENDAHULUAN

1.1 Deskripsi Studi Kasus

Perpustakaan kampus membutuhkan program sederhana untuk membantu pengelolaan koleksi buku, lokasi penyimpanan, antrean mahasiswa, serta pencatatan peminjaman. Program ini dibuat menggunakan bahasa Java dengan menu interaktif yang menerima masukan pengguna melalui *Scanner*. Program menerapkan enam struktur data, yaitu *array* satu dimensi, *array* dua dimensi, *array* tiga dimensi, *ArrayList*, *Stack*, dan *Queue*.

Perpustakaan kampus merupakan fasilitas yang menyediakan berbagai buku untuk menunjang kegiatan belajar mahasiswa. Agar pengelolaan koleksi dan proses peminjaman lebih teratur, diperlukan program yang dapat membantu petugas mengelola kategori buku, data buku, lokasi penyimpanan, antrean mahasiswa, serta riwayat transaksi peminjaman.

Program Sistem Manajemen Perpustakaan Kampus dibuat menggunakan bahasa pemrograman Java dengan menu interaktif yang menerima masukan pengguna melalui *Scanner*. Program menerapkan enam struktur data, yaitu *array* satu dimensi, *array* dua dimensi, *array* tiga dimensi, *ArrayList*, *Queue*, dan *Stack*. Pengguna dapat melihat kategori, mengelola daftar buku, menentukan lokasi penyimpanan, mengatur antrean mahasiswa, serta melihat atau membatalkan transaksi sesuai fungsi yang tersedia.

1.2 Tujuan

Adapun tujuan dari program ini, yaitu:

1. Menerapkan enam struktur data dalam satu program Jawa.
2. Menyediakan menu untuk melihat kategori dan mengelola daftar buku.
3. Mencatat lokasi buku berdasarkan lantai, rak, dan posisi.
4. Mengatur antrean mahasiswa berdasarkan urutan kedatangan.
5. Mencatat transaksi peminjaman dan menyediakan fitur pembatalan transaksi.

BAB II
PERANCANGAN DAN PENERAPAN STRUKTUR DATA

2.1 Array 1 Dimensi (Array 1D)

Array satu dimensi merupakan struktur data linier yang digunakan untuk menyimpan sekumpulan nilai bertipe sama dalam satu urutan indeks. Pada program ini, *array* kategoriBuku bertipe *String* menyimpan empat kategori buku, yaitu Pemrograman, Matematika, Jaringan Komputer, dan Basis Data.

Indeks *array* dimulai dari 0, sehingga Pemrograman berada pada indeks 0 dan Basis Data pada indeks 3. Ketika ditampilkan di menu 1, indeks ditambah 1 agar nomor yang dilihat pengguna dimulai dari 1. *Array* ini juga dipakai pada menu 5 sebagai pilihan kategori dan sebagai sumber nama kategori pada teks transaksi.

2.2 Array 2 Dimensi (Array 2D)

Array dua dimensi *statPinjam* bertipe *int*, dimana pada bagian baris mewakili empat kategori buku dan bagian kolom mewakili tujuh hari dalam seminggu. Nilai pada setiap sel adalah jumlah peminjaman untuk kategori dan hari tersebut. Ketika peminjaman berhasil dicatat, nilai sel yang sesuai bertambah satu melalui *method statPinjam*. Selain itu, seluruh sel yang bernilai 0 pada awal program dikarenakan *array* int diinisialisasi dengan nilai *default* Java.

2.3 Array 3 Dimensi (Array 3D)

Array tiga dimensi lokasiBuku bertipe *String* dengan pembagian, tiga lantai, lima rak pada setiap lantai, dan sepuluh posisi pada setiap rak, sehingga tersedia 3 x 5 x 10 = 150 slot penyimpanan. Setiap slot berisi nama buku atau *null* jika masih kosong.

Pada menu 3, pengguna memasukkan nama buku beserta indeks lantai, rak, dan posisi. Program memeriksa bahwa ketiga indeks berada dalam batas *array* sebelum menyimpan nama buku, sehingga tidak terjadi *ArrayIndexOutOfBoundsException*.

2.4 ArrayList

ArrayList daftarBuku menyimpan judul buku secara dinamis. Berbeda dengan *array* yang ukurannya ditentukan saat dibuat, *ArrayList* dapat bertambah atau berkurang melalui *method add*() dan *remove*(). *Method add*() menambahkan buku di akhir daftar. *Method remove*() menghapus buku dengan judul yang sama dan mengembalikan nilai *boolean*, *true* jika buku ditemukan dan *false* jika tidak.

Nilai ini dipakai program untuk menentukan pesan yang ditampilkan. Isi yang ada dalam *ArrayList* ditampilkan dengan mencetak objeknya langsung sehingga muncul dalam format [buku1, buku2, dst].

2.5 Stack (LIFO)

Stack menerapkan prinsip LIFO (*Last In, First Out*), yaitu data yang terakhir dimasukkan menjadi data pertama yang diambil. *Stack* riwayatTransaksi menyimpan teks setiap transaksi peminjaman melalui *push*().

Pada menu 6, program mengambil transaksi dari puncak *Stack* satu per satu menggunakan *pop*(), sehingga transaksi terbaru diperiksa lebih dulu. Transaksi pertama yang cocok dengan nama yang dicari dibatalkan, sedangkan transaksi lain dipindahkan ke *Stack* sementara *tempStack* lalu dikembalikan ke *Stack* riwayat dengan urutan semula.

2.6 Queue (FIFO)

Queue menerapkan prinsip FIFO (*First In, First Out*), yaitu data yang masuk lebih dahulu akan keluar lebih dahulu. Pada program ini, antrean mahasiswa disimpan dalam *Queue* antrianMhs yang diimplementasikan dengan *LinkedList*. *Method add*() memasukkan mahasiswa ke bagian belakang antrean, *poll*() mengambil sekaligus menghapus mahasiswa di bagian depan, dan *isEmpty*() memeriksa apakah antrean kosong sebelum pemanggilan dilakukan.

2.7 Hubungan Keenam Struktur Data

Keenam struktur data dijalankan dalam satu program yang sama melalui menu utama. *Array* 1D menyediakan kategori yang dipakai oleh menu 1 dan menu 5. Pada menu 5, satu peminjaman yang berhasil mengubah *Array* 2D (statistik) sekaligus

menambah data ke *Stack* (riwayat), dan nama kategori pada teks transaksi diambil dari *Array* 1D.

Stack kemudian dipakai oleh menu 6 untuk pembatalan. *ArrayList* menyimpan daftar buku, *Array* 3D menyimpan lokasi buku berupa nama buku, dan *Queue* mengatur urutan mahasiswa yang menunggu dilayani. Dalam program ini, *ArrayList*, *Array* 3D, dan *Queue* dikelola melalui menu masing-masing dengan data berupa teks yang diketik pengguna.

BAB III
PENJELASAN SOURCE CODE

3.1 Package, Import, dan Method Main

Baris *package com.mycompany.sistemperpustakaan* menentukan paket tempat *class* berada, sesuai struktur *project NetBeans*. Lima perintah *import* memasukkan *class* yang dibutuhkan, yaitu *ArrayList* untuk daftar buku, *LinkedList* sebagai implementasi *Queue*, *Queue* untuk antrean, *Scanner* untuk membaca masukan *keyboard*, dan *Stack* untuk riwayat transaksi.

Class public SistemPerpustakaan memuat *method* main sebagai titik awal program. Di dalam main, objek *Scanner* dibuat dengan bentuk *try-with-resources* sehingga otomatis ditutup ketika blok *try* selesai.

3.2 Deklarasi Struktur Data dan Variabel

Seluruh struktur data dideklarasikan pada bagian awal *blok try*. *Array* kategoriBuku berisi empat kategori. *Array statPinjam* berukuran 4 x 7 untuk statistik, sedangkan *Array* lokasiBuku berukuran 3 x 5 x 10 untuk lokasi. Penggunaan *ArrayList* daftarBuku untuk menampung judul buku, *Stack* riwayatTransaksi untuk menampung riwayat transaksi, dan *Queue* antrianMhs untuk menampung antrean mahasiswa.

Selain itu, variabel *boolean* lanjut bernilai *true* digunakan sebagai penanda agar menu terus ditampilkan, dan diubah menjadi *false* ketika pengguna memilih keluar.

3.3 Perulangan Menu Utama dan Switch

Method perulangan *while* (lanjut) digunakan untuk menampilkan judul dan tujuh pilihan menu pada saat program dijalankan. Perintah *print* digunakan untuk mencetak "Pilih menu :" tanpa harus pindah baris. Adapun variabel pilihan membaca angka menu melalui *nextInt* (), kemudian *nextLine*() dipanggil untuk membuang karakter *Enter* yang tersisa, dan struktur *switch* dengan tanda panah digunakan untuk mengarahkan program ke blok *case* sesuai nilai pilihan, dengan blok *default* untuk angka di luar rentang.

3.4 Menu 1 dan Menu 2

Pada *Case 1*, menampilkan kategori dengan perulangan *for* dari indeks 0 sampai panjang *array* dikurangi satu, dan mencetak (i + 1) agar nomor dimulai dari 1. Sedangkan pada *Case* 2, program mengelola *ArrayList*.

Program membaca pilihan aksi sebagai *String* lalu memakai *if-else* untuk menjalankan aksi 1 yaitu, membaca judul dan memanggil method *add*(), aksi 2 dengan mencetak seluruh isi daftar, dan juga aksi 3 yaitu, membaca judul lalu memanggil method *remove*() yang hasil *boolean*-nya menentukan pesan berhasil atau tidak ditemukan. Untuk aksi di luar 1 sampai 3 dapat menghasilkan pesan pilihan salah.

3.5 Menu 3: Lokasi Buku

Case 3 dimulai dengan membaca nama buku, lalu memulai variabel penghitung kesalahan yaitu *salahInputLokasi* dari 0. Digunakan juga perulangan *while* yang berjalan selama kesalahan masih kurang dari 3. Pada tiap putaran, program dapat membaca indeks lantai, rak, dan posisi, lalu memeriksa bahwa lantai berada di 0 sampai 2, rak di 0 sampai 4, dan posisi di 0 sampai 9.

Apabila valid, nama buku akan disimpan pada *lokasiBuku*[lantai][rak][posisi], kemudian lokasinya dicetak, dan perintah *break* untuk menghentikan perulangan. Jika tidak valid, penghitung bertambah dan pesan kesalahan akan dicetak oleh program. Jika kesalahan yang terjadi sudah 3 kali, maka program akan kembali ke menu utama.

3.6 Menu 4: Antrean Mahasiswa

Case 4 dilakukan dengan membaca aksi antrean lalu memakai *switch* bersarang. Pada aksi 1, program akan membaca nama mahasiswa dan memasukkannya ke bagian belakang antrean dengan method *add*(). Selanjutnya pada aksi 2, program akan memeriksa *isEmpty*(), jika antrean sudah terisi, maka *method poll*() akan mengambil dan menghapus mahasiswa terdepan lalu menampilkan namanya. Sebaliknya apabila antrean kosong, maka program akan menampilkan pesan antrean kosong sehingga tidak terjadi kesalahan. Untuk aksi lain di luar pilihan aksi 1 dan 2, akan menghasilkan pesan pilihan salah.

3.7 Menu 5: Catat Peminjaman dan Statistik

Pada *Case* 5, program akan membaca nama peminjam dan judul buku, lalu masuk ke perulangan *while* dengan penghitung kesalahan *salahInputPinjam* maksimal tiga kali. Selanjutnya program akan menampilkan daftar kategori dari *Array* 1D, membaca nomor kategori ke variabel *kat*, menampilkan daftar hari, dan membaca nomor hari ke variabel hari.

Jika *kat* berada di antara 0 sampai 3 dan hari berada di antara 0 sampai 6, maka program akan menjalankan *statPinjam*, lalu menyusun teks transaksi dengan penggabungan *String* dan menyimpannya ke *Stack* dengan *push*(). Program kemudian mencetak total peminjaman pada sel tersebut dan menghentikan perulangan. Apabila tidak valid, maka penghitung kesalahan akan bertambah dan pengguna diminta mengulang hingga tiga kali.

3.8 Menu 6: Pembatalan Transaksi

Selanjutnya, *Case* 6 hanya akan berjalan jika *Stack* tidak kosong. Program akan membaca nama peminjam yang akan dibatalkan, lalu membuat *Stack* sementara yaitu *tempStack*, dan variabel ketemu bernilai *false*. Pada perulangan *while*, digunakan untuk mengambil transaksi dengan *pop*() dari puncak *Stack*.

Jika teks transaksi diawali dengan nama yang dicari (tanpa membedakan huruf besar dan kecil) dan belum ada transaksi yang dibatalkan, maka transaksi tersebut akan dibuang dan ketemu menjadi bernilai *true*. Selain itu, transaksi akan dipindahkan ke *tempStack*. Pada perulangan yang kedua, program akan mengembalikan isi *tempStack* ke *Stack* riwayat sehingga urutan yang semula pulih. Dikarenakan *pop*() dimulai dari puncak, maka transaksi yang dibatalkan adalah transaksi terbaru atas nama tersebut.

3.9 Menu 7 dan Default

Terakhir yaitu *Case* 7, digunakan untuk mencetak ucapan terima kasih dan mengubah lanjut menjadi *false* sehingga perulangan utama berhenti dan *Scanner* ditutup otomatis. Sedangkan, pada blok *default* akan amenampilkan pesan bahwa pilihan menu tidak ada, lalu program kembali menampilkan menu.

Tabel 3. 1 *Ringkasan Variabel, Perintah, dan Alur*

Variabel/Perintah	Fungsi	Contoh penggunaan
String[]	Menyimpan kumpulan teks berukuran tetap	kategoriBuku
int[][] dan String[][][]	Menyimpan data dalam bentuk tabel dan kubus (dua dan tiga indeks)	statPinjam, lokasiBuku
ArrayList	Daftar dinamis, yaitu *add*() dan *remove*()	daftarBuku
Queue (LinkedList)	Antrean FIFO, yaitu *add*(), *poll*(), dan *isEmpty*()	antrianMhs
Stack	Tumpukan LIFO, yaitu *push*() dan *pop*()	riwayatTransaksi, tempStack
while	Mengulang selama kondisi benar	Menu utama, validasi input
for	Mengulang berdasarkan indeks	Menampilkan kategori dan hari
if-else	Memilih tindakan berdasarkan kondisi	Validasi lokasi, kategori, dan hari
switch-case	Memilih cabang berdasarkan nilai	Menu utama dan menu antrean
break	Menghentikan perulangan atau cabang	Keluar dari while saat input valid
nextInt() dan nextLine()	Membaca angka dan satu baris teks	pilihan, nama buku
boolean	Penanda benar atau salah	lanjut, ketemu

3.10 Rincian Kode

Pada bagian ini, penjelasan source code akan diuraikan berdasarkan nomor baris pada *file* SistemPerpustakaan.java. Baris kosong dalam program hanya merapikan tampilan, sedangkan simbol kurung kurawal pembuka dan penutup menandai awal dan akhir blok. Penjelasan berikut mencakup deklarasi, variabel, *method*, perulangan, percabangan, dan proses pengolahan data.

Tabel 3. 2 *Penjelasan Kode Pada SistemPerpustakaan.java*

Baris	Kode/Perintah	Penjelasan
1	package com.mycompany.sistemperpustakaan;	Menetapkan paket tempat *class* berada.
3	import java.util.ArrayList;	Mengimpor *ArrayList* untuk daftar buku dinamis.
4	import java.util.LinkedList;	Mengimpor *LinkedList* yang dipakai sebagai implementasi *Queue*.
5	import java.util.Queue;	Mengimpor *interface Queue* untuk antrean FIFO.
6	import java.util.Scanner;	Mengimpor *Scanner* untuk membaca masukan *keyboard*.
7	import java.util.Stack;	Mengimpor *Stack* untuk riwayat transaksi LIFO.
9	public class SistemPerpustakaan {	Mendeklarasikan class utama. Kurung kurawal membuka isi *class*.
10	public static void main(String[] args) {	*Method* utama yang dijalankan pertama kali. *void* berarti tidak mengembalikan nilai.
12	try (Scanner input = new Scanner(System.in)) {	Membuat objek *Scanner* bernama *input* untuk membaca *keyboard*. Bentuk *try-with-resources* akan menutup *Scanner* otomatis setelah blok selesai.
14	String[] kategoriBuku = {"Pemrograman", "Matematika", "Jaringan Komputer", "Basis Data"};	*Array* 1D, merupakan *array String* yang berisi empat kategori dengan indeks 0 sampai 3.
15	int[][] statPinjam = new int[4][7]; // 4 kategori, 7 hari	*Array* 2D, untuk tabel bilangan bulat 4 baris (kategori) x 7 kolom (hari). Semua sel bernilai awal 0.
16	String[][][] lokasiBuku = new String[3][5][10]; // Lantai, Rak, Posisi	*Array* 3D, digunakan untuk menyimpan nama buku berdasarkan lantai, rak, dan posisi dengan total 150 slot, yang dimana slot kosong bernilai *null*.

Baris	Kode/Perintah	Penjelasan
18	ArrayList daftarBuku = new ArrayList<>();	*Arraylist,* digunakan untuk daftar kosong untuk menampung judul buku dan ukurannya menyesuaikan jumlah data.
19	Stack riwayatTransaksi = new Stack<>();	*Stack,*berisi tumpukan kosong untuk riwayat transaksi (LIFO).
20	Queue antrianMhs = new LinkedList<>();	*Queue*, berisi antrean kosong dengan implementasi *LinkedList* (FIFO).
22	boolean lanjut = true;	Variabel penanda, selama bernilai *true*, maka menu utama akan terus diulang.
24	while (lanjut) {	Perulangan utama program, akan berhenti saat lanjut menjadi *false*.
25	System.out.println("\n=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===");	Mencetak judul menu. Simbol *\n* menambah baris kosong sebelum judul.
26-32	System.out.println("1. Tampilkan Kategori Buku"); ..... System.out.println("7. Keluar");	Mencetak tujuh pilihan menu, masing-masing dengan *println* sehingga berada di baris sendiri.
33	System.out.print("Pilih menu : ");	Mencetak kode tanpa pindah baris agar masukan pengguna berada di sebelahnya.
35	int pilihan = input.nextInt();	Membaca angka menu dan menyimpannya di variabel pilihan.
36	input.nextLine();	Membuang karakter *Enter* yang tersisa setelah *nextInt*() agar pembacaan teks berikutnya tidak terlewat.
38	switch (pilihan) {	Percabangan *switch* memilih blok *case* sesuai nilai pilihan. Tanda panah membuat setiap *case* berdiri sendiri.
39-40	case 1 -> { System.out.println("\n-- Kategori Buku --");	*Case* 1 menampilkan kategori buku. Baris kedua mencetak judul bagian.
41-44	for (int i = 0; i < kategoriBuku.length; i++) { System.out.println((i + 1) + ". " + kategoriBuku[i]);	Perulangan *for* dari i = 0 sampai panjang *array*. Mencetak nomor (i + 1) agar dimulai dari 1, diikuti nama kategori pada indeks i.

Baris	Kode/Perintah	Penjelasan
	} }	

45-49	case 2 -> {
	... println judul dan submenu 1. Tambah, 2. Tampilkan, 3. Hapus
	};
		Case 2 mengelola *ArrayList*. Mencetak judul dan tiga pilihan aksi.

50-51	System.out.print("Pilih aksi: "); String aksiBuku = input.nextLine();	Membaca pilihan aksi sebagai *String* dan menyimpannya di *aksiBuku*.
54	if (aksiBuku.equals("1")) {	Jika aksi sama dengan 1, blok tambah buku dijalankan. Perbandingan *String* memakai *equals*(), bukan ==.
55-58	System.out.print("Masukkan nama buku: "); String bukuBaru = input.nextLine(); daftarBuku.add(bukuBaru); System.out.println("Sip, buku berhasil ditambah.");	Meminta judul buku, menyimpannya ke *bukuBaru*, lalu *add*() menambahkannya di akhir *ArrayList* dan mencetak pesan berhasil.
59-60	} else if (aksiBuku.equals("2")) { System.out.println("Koleksi Buku saat ini: " + daftarBuku);	Jika aksi 2, seluruh isi *ArrayList* dicetak dalam format [buku1, buku2, ...].
61-63	} else if (aksiBuku.equals("3")) { System.out.print("Masukkan nama buku yang mau dihapus: "); String bukuHapus = input.nextLine();	Jika aksi 3, meminta judul buku yang akan dihapus dan menyimpannya di *bukuHapus*.

64-68	if (daftarBuku.remove(bukuHapus)) {
	...berhasil dihapus...
	} else {
	...tidak ketemu...
	}
		remove() menghapus buku pertama yang judulnya sama dan mengembalikan *true* jika ditemukan, *false* jika tidak. Hasilnya langsung dipakai sebagai kondisi *if*. Pencarian membedakan huruf besar dan kecil.

69-71	} else {
	System.out.println("Pilihan salah!");
	}
		Jika aksi bukan 1, 2, atau 3, tampil pesan pilihan salah.

Baris	Kode/Perintah	Penjelasan
74-77	case 3 -> { ... println judul ... System.out.print("Masukkan nama buku: "); String namaBuku = input.nextLine();	*Case* 3 menyimpan lokasi buku ke *Array* 3D. Program meminta nama buku dan menyimpannya di *namaBuku*.
79-80	int salahInputLokasi = 0; while (salahInputLokasi < 3) {	Penghitung kesalahan dimulai dari 0. Perulangan *while* berjalan selama kesalahan kurang dari 3, sehingga pengguna punya maksimal tiga kesempatan.
81-86	System.out.print("Lantai (0-2): "); int lantai = input.nextInt(); ... rak ... ... posisi ...	Membaca tiga angka sebagai indeks *array* 3D yaitu lantai, rak, dan posisi.
88-89	// cek validasi lokasi if (lantai >= 0 && lantai < 3 && rak >= 0 && rak < 5 && posisi >= 0 && posisi < 10) {	Validasi untuk seluruh indeks harus berada dalam batas *array*. Operator && berarti semua syarat harus benar. Hal ini untuk mencegah ArrayIndexOutOfBoundsException.
90	lokasiBuku[lantai][rak][posisi] = namaBuku;	Menyimpan nama buku pada slot *array* 3D yang dipilih.
91-92	System.out.println("Buku '" + namaBuku + "' disimpen di Lantai " + ...); break;	Mencetak lokasi tempat buku disimpan. *break* menghentikan *while* karena *input* sudah benar.

93-95	} else {
	salahInputLokasi++;
	System.out.println("Lokasi nggak valid!");
		Jika tidak valid, penghitung kesalahan bertambah satu (++) dan pesan kesalahan dicetak.

96-100	if (salahInputLokasi == 3) {
	...kembali ke menu utama...
	} else {
	..."Coba lagi."
	}
		Jika kesalahan sudah tiga kali, pengguna diberi tahu akan kembali ke menu utama (*while* berhenti karena syaratnya tidak terpenuhi). Jika belum, pengguna diminta mengulang.

105-110	case 4 -> { ... println judul dan submenu ...	*Case* 4 mengelola *Queue*. Mencetak sub-menu tambah mahasiswa dan panggil

Baris	Kode/Perintah	Penjelasan
	String aksiAntrean = input.nextLine();	mahasiswa, lalu membaca pilihan ke *aksiAntrean*.
112	switch (aksiAntrean) {	*Switch* bersarang untuk memilih aksi antrean berdasarkan teks.
113-118	case "1" -> { System.out.print("Masukkan nama mahasiswa: "); String namaMhs = input.nextLine(); antrianMhs.add(namaMhs); System.out.println(namaMhs + " masuk ke antrian."); }	Aksi 1, program akan membaca nama mahasiswa lalu *add*() memasukkannya ke bagian belakang antrean (*enqueue*).
119-125	case "2" -> { if (!antrianMhs.isEmpty()) { ... "Melayani mahasiswa: " + antrianMhs.poll() } else { ... "Antrian lagi kosong." } }	Aksi 2, menggunakan *isEmpty*() untuk memeriksa antrean kosong, penggunaan tanda seru untuk membaliknya sehingga benar bila antrean telah terisi. Jika terisi, maka *method poll*() mengambil dan menghapus mahasiswa terdepan (*dequeue*, FIFO). Jika kosong, maka pesan antrean kosong akan dicetak.
126	default -> System.out.println("Pilihan salah.");	Menangani aksi antrean selain 1 dan 2.
130-136	case 5 -> { ... println judul ... String namaPeminjam = input.nextLine(); ... String judulDipinjam = input.nextLine();	*Case* 5 mencatat peminjaman. Membaca nama peminjam dan judul buku ke variabel *namaPeminjam* dan *judulDipinjam*.
138-139	int salahInputPinjam = 0; while (salahInputPinjam < 3) {	Penghitung kesalahan peminjaman dan perulangan maksimal tiga kali, sama seperti pada *case* 3.
140-146	System.out.println("\nUntuk data statistik, lengkapi info berikut:");	Mencetak instruksi, lalu *for* menampilkan kategori dari *Array* 1D beserta nomor indeksnya (0 sampai 3) sebagai pilihan.

Baris	Kode/Perintah	Penjelasan

	System.out.println("Kategori Buku:");
	for (int i = 0; i < kategoriBuku.length; i++) {
	System.out.println(i + " : " + kategoriBuku[i]);
	}	

147-148	System.out.print("Pilih kategori (0-4): "); int kat = input.nextInt();	Membaca nomor kategori ke variabel *kat*.

150-154	System.out.println("\nHari Peminjaman:");
	System.out.println("0 : Senin\n1 : Selasa ... 6 : Minggu");
	System.out.print("Pilih hari (0-6): ");
	int hari = input.nextInt();
	input.nextLine();
		Mencetak daftar hari (\n memecah baris) dan membaca nomor hari ke variabel hari. *Method input.nextLine*() membuang sisa Enter.

156	if (kat >= 0 && kat < 4 && hari >= 0 && hari < 7) {	Validasi kategori harus 0 sampai 3 dan hari 0 sampai 6, sesuai ukuran *array statPinjam*[4][7].
157	statPinjam[kat][hari]++;	Menambah satu pada sel *Array* 2D untuk kategori dan hari yang dipilih, sebagai pencatatan statistik.
158-159	String transaksi = namaPeminjam + " meminjam '" + judulDipinjam + "' (Kategori: " + kategoriBuku[kat] + ")"; riwayatTransaksi.push(transaksi);	Menyusun teks transaksi dengan penggabungan *String* (+), memakai nama kategori dari *Array* 1D. Sedangkan, *push*() menaruhnya di puncak *Stack* sebagai transaksi terbaru.
161-163	System.out.println("\nSip, transaksi berhasil dicatat!"); System.out.println("Total peminjaman kategori " + ... + statPinjam[kat][hari]); break;	Mencetak pesan berhasil dan total peminjaman pada sel tersebut. *break* digunakan untuk menghentikan *while* karena *input* sudah benar.
164-172	} else { salahInputPinjam++;	Jika *input* tidak valid, penghitung kesalahan bertambah dan pesan kesalahan

Baris	Kode/Perintah	Penjelasan
	System.out.println("\n[!] Input kategori atau hari salah!"); if (salahInputPinjam == 3) {...} else {...} }	dicetak. Setelah tiga kali salah, pengguna kembali ke menu utama. Apabila belum melampau limit yang ada, maka pengguna akan diminta mengulang.

172-174	case 6 -> {
	System.out.println("\n-- Batalkan Transaksi Berdasarkan Nama --");
	if (!riwayatTransaksi.isEmpty()) {
		Case 6 membatalkan transaksi dari *Stack*. Pertama, progam akan memeriksa apakah *Stack* telah terisi. Apabila kosong, maka bagian *else* akan dijalankan.

175-176	System.out.print("Masukkan nama peminjam yang ingin dibatalkan: "); String namaBatal = input.nextLine();	Meminta nama peminjam yang transaksinya akan dibatalkan.
179-180	Stack tempStack = new Stack<>(); boolean ketemu = false;	*tempStack* adalah *Stack* sementara untuk menampung transaksi yang tidak dibatalkan. Variabel *ketemu* menandai apakah transaksi yang dicari sudah ditemukan.
183-184	while (!riwayatTransaksi.isEmpty()) { String transaksi = riwayatTransaksi.pop();	Perulangan mengosongkan *Stack* riwayat satu per satu, *method pop*() mengambil transaksi dari puncak (yang terbaru lebih dulu, menggunakan sifat LIFO).

186-188	if (transaksi.toLowerCase().startsWith (namaBatal.toLowerCase() + " ") && !ketemu) {
	... "Berhasil membatalkan transaksi: " + transaksi
	ketemu = true;
		Memeriksa apakah transaksi diawali nama yang sedang dicari. *Method toLowerCase*() digunakan untuk menyamakan huruf besar dan kecil. Selanjutnya, spasi ditambahkan agar nama pendek tidak cocok dengan nama lain yang diawali huruf sama. Adapun syarat *!ketemu* memastikan hanya transaksi terbaru yang dibatalkan. Jika cocok, maka transaksi akan dibuang (tidak dikembalikan) dan ketemu menjadi *true*.

190-193	} else {
	tempStack.push(transaksi);
	}
		Jika tidak cocok (atau sudah ada yang dibatalkan), maka transaksi akan dipindahkan ke *tempStack*.

Baris	Kode/Perintah	Penjelasan

195-197	while (!tempStack.isEmpty()) {
	riwayatTransaksi.push(tempStack.pop());
	}
		Mengembalikan isi *tempStack* ke *Stack* riwayat, dengan membalikkan urutan sebanyak dua kali sehingga urutan transaksi kembali seperti semula.

199-201	if (!ketemu) {
	System.out.println("Transaksi atas nama '" + namaBatal + "' nggak ketemu tuh.");
	}
		Jika tidak ada transaksi yang cocok, program akan mencetak pesan tidak ditemukan.

203-207	} else {
	System.out.println("Nggak ada riwayat transaksi nih.");
	}
	break;
		Merupakan bagian *else* dari pemeriksaan awal, pada bagian *Stack* kosong sehingga tidak ada yang dapat dibatalkan. Lalu, kode *break* digunakan untuk mengakhiri *case* 6.

209-213	case 7 -> {
	System.out.println("\nTerima kasih udah pakai sistem ini!");
	lanjut = false;
	break;
	}
		Case 7, program akan mencetak ucapan terima kasih dan mengubah lanjut menjadi *false* sehingga *while* utama berhenti dan program selesai.

215-217	default -> {
	System.out.println("\nPilihan menu nggak ada. Coba angka yang bener.");
	break;
	}
		Dijalankan apabila angka menu bukan di antara pilihan 1 sampai 7, kemudian program akan menampilkan pesan lalu kembali ke menu.

218-223	} } } } }	Kurung kurawal penutup untuk *switch*, *while*, *try* (*Scanner* otomatis ditutup), *method main*, dan *class* SistemPerpustakaan.

BAB IV
DOKUMENTASI DAN HASIL PENGUJIAN

4.1. Menampilkan Array 1 Dimensi (Kategori Buku)

Pada bagian ini, program menampilkan isi dari *Array* 1 Dimensi *kategoriBuku* menggunakan perulangan *for*. Perulangan digunakan untuk mengakses setiap elemen *array* berdasarkan indeksnya, kemudian menampilkan seluruh kategori buku yang tersimpan, seperti Pemrograman, Matematika, Jaringan Komputer, dan Basis Data.

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir
7. Keluar
Pilih menu : 1

-- Kategori Buku --
1. Pemrograman
2. Matematika
3. Jaringan Komputer
4. Basis Data
```

Gambar 4. 1 *Tampilan Array 1 Dimensi (Kategori Buku)*

4.2. Memasukkan Data ke ArrayList (Tambah Buku)

Pada bagian ini, pengguna memilih menu tambah buku untuk memasukkan data buku baru ke dalam *ArrayList*. Program meminta pengguna memasukkan informasi buku sesuai dengan *input* yang tersedia. Setelah data dimasukkan, program menyimpan data tersebut ke dalam *ArrayList* sehingga jumlah koleksi buku dapat bertambah secara dinamis.

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir
7. Keluar
Pilih menu : 2

-- Kelola Daftar Buku --
1. Tambah Buku
2. Tampilkan Buku
3. Hapus Buku
Pilih aksi: 1
Masukkan nama buku: belajar java
buku berhasil ditambah.
```

Gambar 4. 2 *Tampilan Penambahan Data Buku ke ArrayList*

4.3. Menampilkan Isi ArrayList (Daftar Buku)

Pada bagian ini, program menampilkan seluruh data buku yang tersimpan di dalam *ArrayList* setelah proses penambahan data dilakukan. Program menggunakan perulangan untuk mengakses setiap elemen *ArrayList* dan mencetak informasi buku ke layar. Dengan demikian, pengguna dapat melihat daftar koleksi buku yang tersedia, termasuk buku yang baru saja ditambahkan.

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir
7. Keluar
Pilih menu : 2

-- Kelola Daftar Buku --
1. Tambah Buku
2. Tampilkan Buku
3. Hapus Buku
Pilih aksi: 2
Koleksi Buku saat ini: [belajar java]
```

Gambar 4. 3 *Tampilan Isi ArrayList (Daftar Buku)*

4.4. Menampilkan Array 3 Dimensi (Lokasi Buku)

Pada bagian ini, pengguna memasukkan indeks lantai, nomor rak, dan posisi buku untuk menentukan lokasi penyimpanan. Program kemudian menggunakan ketiga indeks tersebut untuk mengakses dan mengisi data pada *Array* 3 Dimensi sesuai dengan lokasi yang dipilih.

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir
7. Keluar
Pilih menu : 3

-- Lokasi Buku --
Masukkan nama buku: buku PBO
Lantai (0-2): 1
Rak (0-4): 2
Posisi (0-9): 5
Buku 'buku PBO ' disimpen di Lantai 1, Rak 2, Posisi 5
```

Gambar 4. 4 *Tampilan Pengisian Data Array 3 Dimensi (Lokasi Buku)*

4.5. Tampilan Queue (Mengelola Antrean Mahasiswa)

Pada bagian ini, pengguna memasukkan nama mahasiswa yang ingin meminjam buku ke dalam antrean. Program menambahkan nama mahasiswa tersebut ke *Queue* menggunakan *method offer*(). Data mahasiswa dimasukkan sesuai urutan kedatangan sehingga mahasiswa yang lebih dahulu masuk akan dilayani terlebih dahulu berdasarkan prinsip FIFO (*First In First Out*).

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir
7. Keluar
Pilih menu : 4

-- Antrian Mahasiswa --
1. Tambah Mahasiswa ke Antrian
2. Panggil Mahasiswa Berikutnya
Pilih aksi: 1
Masukkan nama mahasiswa: carla
carla masuk ke antrian.
```

Gambar 4. 5 *Tampilan Penambahan Data ke Queue (Antrean Mahasiswa)*

Pada bagian ini, program memanggil data mahasiswa yang berada di posisi paling depan dalam antrean. Program menggunakan *method poll*() untuk mengambil sekaligus menghapus mahasiswa terdepan dari *Queue*. Proses ini menerapkan prinsip FIFO (*First In First Out*), yaitu mahasiswa yang pertama masuk ke antrean akan diproses terlebih dahulu.

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir
7. Keluar
Pilih menu : 4

-- Antrian Mahasiswa --
1. Tambah Mahasiswa ke Antrian
2. Panggil Mahasiswa Berikutnya
Pilih aksi: 2
Melayani mahasiswa: carla
```

Gambar 4. 6 *Tampilan Pemanggilan Data dari Queue (Antrean Mahasiswa)*

4.6. Menampilkan Array 2 Dimensi dan Stack (Pencatatan Peminjaman Buku)

Pada bagian ini, pengguna memasukkan data peminjaman buku berdasarkan kategori dan hari. Program menyimpan jumlah peminjaman ke dalam *Array* 2 Dimensi sesuai indeks kategori dan hari yang dipilih. Selain itu, program menambahkan informasi transaksi peminjaman ke dalam *Stack* menggunakan *method push*() sebagai riwayat transaksi. Dengan demikian, data peminjaman tercatat dalam *Array* 2 Dimensi, sedangkan riwayat transaksi disimpan dalam *Stack* dengan prinsip LIFO (*Last In First Out*).

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir (Undo)
7. Keluar
Pilih menu : 5

-- Catat Peminjaman --
Masukkan nama peminjam: carla
Masukkan judul buku yang dipinjam: jaringan komputer

Untuk data statistik, lengkapi info berikut:
Kategori Buku:
0 : Pemrograman
1 : Matematika
2 : Jaringan Komputer
3 : Basis Data
Pilih kategori (0-4): 2

Hari Peminjaman:
0 : Senin
1 : Selasa
2 : Rabu
3 : Kamis
4 : Jumat
5 : Sabtu
6 : Minggu
Pilih hari (0-6): 2

Sip, transaksi berhasil dicatat!
Total peminjaman kategori Jaringan Komputer di hari ke-2 : 1
```

Gambar 4. 7 *Tampilan Pencatatan Peminjaman Buku Menggunakan Array 2 Dimensi dan Stack*

4.7. Tampilan Stack (Undo Peminjaman Buku)

Pada bagian ini, pengguna menjalankan fitur *undo* untuk membatalkan pencatatan peminjaman terakhir. Program menggunakan *method pop*() untuk menghapus transaksi yang berada di posisi teratas *Stack*. Proses ini menerapkan prinsip LIFO (*Last In First Out*), yaitu transaksi yang terakhir dicatat akan dihapus terlebih dahulu. Dengan fitur ini,

pengguna dapat membatalkan transaksi peminjaman terakhir yang tersimpan dalam riwayat *Stack*.

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir (Undo)
7. Keluar
Pilih menu : 6

-- Batalkan Transaksi Berdasarkan Nama --
Masukkan nama peminjam yang ingin dibatalkan: carla
Berhasil membatalkan transaksi: carla meminjam 'jaringan komputer' (Kategori: Jaringan Komputer)
```

Gambar 4. 8 *Tampilan Penghapusan Data dari Stack (Undo Peminjaman)*

4.8. Tampilan Validasi Tiga Kali Kesalahan Input pada Fitur Peminjaman

Pada bagian ini, dilakukan pengujian terhadap validasi *input* pada fitur peminjaman buku. Pengguna memasukkan data yang tidak sesuai sebanyak tiga kali untuk mengetahui apakah program dapat mendeteksi kesalahan *input* dan menjalankan mekanisme validasi yang telah dibuat. Pengujian ini bertujuan untuk memastikan bahwa program dapat menangani kesalahan *input* sesuai dengan ketentuan yang diterapkan pada *source code*.

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir (Undo)
7. Keluar
Pilih menu : 5

-- Catat Peminjaman --
Masukkan nama peminjam: carla
Masukkan judul buku yang dipinjam: 5

Untuk data statistik, lengkapi info berikut:
Kategori Buku:
0 : Pemrograman
1 : Matematika
2 : Jaringan Komputer
3 : Basis Data
Pilih kategori (0-4): 4

Hari Peminjaman:
0 : Senin
1 : Selasa
2 : Rabu
3 : Kamis
4 : Jumat
5 : Sabtu
6 : Minggu
Pilih hari (0-6): 3

[!] Input kategori atau hari salah!
Ayo masukin angka yang bener.
```

Gambar 4. 9 *Validasi Tiga Kali Kesalahan Input pada Fitur Peminjaman*

4.9. Keluar dari Program

Pada bagian ini, pengguna memilih menu keluar untuk mengakhiri program Sistem Manajemen Perpustakaan Kampus. Program menjalankan perintah untuk menghentikan perulangan menu utama sehingga program selesai dijalankan dan pengguna kembali ke terminal atau konsol.

```
=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===
1. Tampilkan Kategori Buku
2. Kelola Daftar Buku
3. Cari/Simpan Lokasi Buku
4. Kelola Antrian Mahasiswa
5. Catat Peminjaman & Statistik
6. Batalkan Transaksi Terakhir (Undo)
7. Keluar
Pilih menu : 7

Terima kasih udah pakai sistem ini!
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  01:23 min
Finished at: 2026-10-25T15:05:02+08:00
------------------------------------------------------------------------
```

Gambar 4. 10 *Tampilan Keluar dari Program*

BAB V
KESIMPULAN

5.1 Kesimpulan

Program Sistem Manajemen Perpustakaan Kampus berhasil menerapkan enam struktur data dalam satu program Java berbasis menu konsol. *Array* satu dimensi digunakan untuk kategori buku, *array* dua dimensi untuk statistik peminjaman per kategori dan hari, *array* tiga dimensi untuk lokasi buku berdasarkan lantai, rak, dan posisi, *ArrayList* untuk daftar buku yang dinamis, *Queue* untuk antrean mahasiswa dengan prinsip FIFO, dan *Stack* untuk riwayat transaksi dengan prinsip LIFO.

Dengan adanya program ini, dapat diketahui bahwa pemilihan struktur data harus disesuaikan dengan sifat datanya, seperti penggunaan *array* untuk data berukuran tetap dan akses berindeks, *ArrayList* untuk data yang jumlahnya berubah, *Queue* untuk urutan kedatangan, dan *Stack* untuk urutan terbalik.

Program juga melatih penggunaan perulangan, percabangan, *switch*, dan validasi *input*. Dengan dilakukan pengujian, hal tersebut menunjukkan bahwa seluruh fitur berjalan sesuai tujuan. Untuk pengembangan berikutnya, program dapat dilengkapi penanganan kesalahan *input* non-angka, sinkronisasi statistik saat transaksi dibatalkan, pemeriksaan lokasi terhadap daftar buku, serta penyimpanan data ke *file* atau *database*.

LAMPIRAN

- Link Github:
  https://github.com/rifqysyahdi/PerpustakaanKampus-Java-Kelompok-24.git

- Source Code Full

```
package com.mycompany.sistemperpustakaan;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;
import java.util.Stack;

public class SistemPerpustakaan {
    public static void main(String[] args) {

        try (Scanner input = new Scanner(System.in)) {

            String[] kategoriBuku = {"Pemrograman", "Matematika", "Jaringan Komputer", "Basis Data"};
            int[][] statPinjam = new int[4][7]; // 4 kategori, 7 hari
            String[][][] lokasiBuku = new String[3][5][10]; // Lantai, Rak, Posisi

            ArrayList<String> daftarBuku = new ArrayList<>();
            Stack<String> riwayatTransaksi = new Stack<>();
            Queue<String> antrianMhs = new LinkedList<>();

            boolean lanjut = true;

            while (lanjut) {
                System.out.println("\n=== SISTEM MANAJEMEN PERPUSTAKAAN KAMPUS ===");
                System.out.println("1. Tampilkan Kategori Buku");
```

```
System.out.println("2. Kelola Daftar Buku");
System.out.println("3. Cari/Simpan Lokasi Buku");
System.out.println("4. Kelola Antrian Mahasiswa");
System.out.println("5. Catat Peminjaman & Statistik");
System.out.println("6. Batalkan Transaksi Terakhir (Undo)");
System.out.println("7. Keluar");
System.out.print("Pilih menu : ");

int pilihan = input.nextInt();
input.nextLine();

switch (pilihan) {
   case 1 -> {
      System.out.println("\n-- Kategori Buku --");
      for (int i = 0; i < kategoriBuku.length; i++) {
         System.out.println((i + 1) + ". " + kategoriBuku[i]);
      }
   }

   case 2 -> {
      System.out.println("\n-- Kelola Daftar Buku --");
      System.out.println("1. Tambah Buku");
      System.out.println("2. Tampilkan Buku");
      System.out.println("3. Hapus Buku");
      System.out.print("Pilih aksi: ");
      String aksiBuku = input.nextLine();

      if (aksiBuku.equals("1")) {
         System.out.print("Masukkan nama buku: ");
         String bukuBaru = input.nextLine();
         daftarBuku.add(bukuBaru);
         System.out.println("Sip, buku berhasil ditambah.");
      } else if (aksiBuku.equals("2")) {
         System.out.println("Koleksi Buku saat ini: " + daftarBuku);
      } else if (aksiBuku.equals("3")) {
         System.out.print("Masukkan nama buku yang mau dihapus: ");
```

```
                    String bukuHapus = input.nextLine();
                    if (daftarBuku.remove(bukuHapus)) {
                        System.out.println("Buku berhasil dihapus.");
                    } else {
                        System.out.println("Buku tidak ketemu.");
                    }
                } else {
                    System.out.println("Pilihan salah!");
                }
            }

            case 3 -> {
                System.out.println("\n-- Lokasi Buku --");
                System.out.print("Masukkan nama buku: ");
                String namaBuku = input.nextLine();

                int salahInputLokasi = 0;
                while (salahInputLokasi < 3) {
                    System.out.print("Lantai (0-2): ");
                    int lantai = input.nextInt();
                    System.out.print("Rak (0-4): ");
                    int rak = input.nextInt();
                    System.out.print("Posisi (0-9): ");
                    int posisi = input.nextInt();

                    // cek validasi lokasi
                    if (lantai >= 0 && lantai < 3 && rak >= 0 && rak < 5 && posisi >= 0 &&
posisi < 10) {
                        lokasiBuku[lantai][rak][posisi] = namaBuku;
                        System.out.println("Buku '" + namaBuku + "' disimpen di Lantai " + lantai
+ ", Rak " + rak + ", Posisi " + posisi);
                        break;
                    } else {
                        salahInputLokasi++;
                        System.out.println("Lokasi nggak valid!");
                        if (salahInputLokasi == 3) {
```

```
                System.out.println("Udah 3 kali salah, kembali ke menu utama.");
            } else {
                System.out.println("Coba lagi.");
            }
        }
    }
}

case 4 -> {
    System.out.println("\n-- Antrian Mahasiswa --");
    System.out.println("1. Tambah Mahasiswa ke Antrian");
    System.out.println("2. Panggil Mahasiswa Berikutnya");
    System.out.print("Pilih aksi: ");
    String aksiAntrean = input.nextLine();

    switch (aksiAntrean) {
        case "1" -> {
            System.out.print("Masukkan nama mahasiswa: ");
            String namaMhs = input.nextLine();
            antrianMhs.add(namaMhs);
            System.out.println(namaMhs + " masuk ke antrian.");
        }
        case "2" -> {
            if (!antrianMhs.isEmpty()) {
                System.out.println("Melayani mahasiswa: " + antrianMhs.poll());
            } else {
                System.out.println("Antrian lagi kosong.");
            }
        }
        default -> System.out.println("Pilihan salah.");
    }
}

case 5 -> {
    System.out.println("\n-- Catat Peminjaman --");
```

```
                System.out.print("Masukkan nama peminjam: ");
                String namaPeminjam = input.nextLine();
                System.out.print("Masukkan judul buku yang dipinjam: ");
                String judulDipinjam = input.nextLine();

                int salahInputPinjam = 0;
                while (salahInputPinjam < 3) {
                    System.out.println("\nUntuk data statistik, lengkapi info berikut:");



                    System.out.println("Kategori Buku:");
                    for (int i = 0; i < kategoriBuku.length; i++) {
                        System.out.println(i + " : " + kategoriBuku[i]);
                    }
                    System.out.print("Pilih kategori (0-4): ");
                    int kat = input.nextInt();

                    System.out.println("\nHari Peminjaman:");
                    System.out.println("0 : Senin\n1 : Selasa\n2 : Rabu\n3 : Kamis\n4 : Jumat\n5
: Sabtu\n6 : Minggu");
                    System.out.print("Pilih hari (0-6): ");
                    int hari = input.nextInt();
                    input.nextLine();

                    if (kat >= 0 && kat < 4 && hari >= 0 && hari < 7) {
                        statPinjam[kat][hari]++;
                        String transaksi = namaPeminjam + " meminjam '" + judulDipinjam + "'
(Kategori: " + kategoriBuku[kat] + ")";
                        riwayatTransaksi.push(transaksi);

                        System.out.println("\nSip, transaksi berhasil dicatat!");
                        System.out.println("Total peminjaman kategori " + kategoriBuku[kat] + "
di hari ke-" + hari + " : " + statPinjam[kat][hari]);
                        break;
                    } else {
                        salahInputPinjam++;
```

```
                    System.out.println("\n[!] Input kategori atau hari salah!");
                    if (salahInputPinjam == 3) {
                        System.out.println("Udah 3 kali salah input!, kembali ke menu utama. <<");
                    } else {
                        System.out.println("Ayo masukin angka yang bener.");
                    }
                }
            }
        }

        case 6 -> {
            System.out.println("\n-- Batalkan Transaksi Berdasarkan Nama --");
            if (!riwayatTransaksi.isEmpty()) {
                System.out.print("Masukkan nama peminjam yang ingin dibatalkan: ");
                String namaBatal = input.nextLine();


                Stack<String> tempStack = new Stack<>();
                boolean ketemu = false;


                while (!riwayatTransaksi.isEmpty()) {
                    String transaksi = riwayatTransaksi.pop();

                    if (transaksi.toLowerCase().startsWith(namaBatal.toLowerCase() + " ") && !ketemu) {
                        System.out.println("Berhasil membatalkan transaksi: " + transaksi);
                        ketemu = true;

                    } else {
                        tempStack.push(transaksi);
                    }
                }

                while (!tempStack.isEmpty()) {
```

```
                        riwayatTransaksi.push(tempStack.pop());
                    }

                    if (!ketemu) {
                        System.out.println("Transaksi atas nama '" + namaBatal + "' nggak ketemu tuh.");
                    }

                } else {
                    System.out.println("Nggak ada riwayat transaksi nih.");
                }
                break;
            }

            case 7 -> {
                System.out.println("\nTerima kasih udah pakai sistem ini!");
                lanjut = false;
                break;
            }

            default -> {
                System.out.println("\nPilihan menu nggak ada. Coba angka yang bener.");
                break;
            }
        }
    }
  }
}
```